

# Autómatas en dos direcciones

Clase 13

IIC2223 / IIC2224

Prof. Cristian Riveros

# Outline

Teorema de Myhill-Nerode (anterior)

2DFA

2DFA vs DFA

# Outline

Teorema de Myhill-Nerode (anterior)

2DFA

2DFA vs DFA

# Relación de equivalencia dada por un DFA (recordatorio)

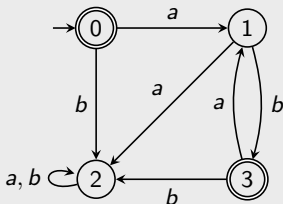
Sea  $L \subseteq \Sigma^*$  un lenguaje regular y  $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$  tal que  $L = \mathcal{L}(\mathcal{A})$ .

## Definición

Se define la **relación de equivalencia**  $\equiv_{\mathcal{A}}$  entre palabras en  $\Sigma^*$  como:

$$u \equiv_{\mathcal{A}} v \iff \hat{\delta}(q_0, u) = \hat{\delta}(q_0, v)$$

## Ejemplo



$$[\epsilon]_{\mathcal{A}} = \epsilon$$

$$[a]_{\mathcal{A}} = a(ba)^*$$

$$[ab]_{\mathcal{A}} = ab(ab)^*$$

$$[b]_{\mathcal{A}} = b\Sigma^* + \Sigma^*(aa + bb)\Sigma^*$$

¿qué lenguaje **acepta** el autómata?

# Relaciones de Myhill-Nerode (recordatorio)

Sea  $L \subseteq \Sigma^*$  cualquier lenguaje.

## Definición (Relación de Myhill-Nerode)

Una relación de equivalencia  $\equiv$  en  $\Sigma^*$  es de **Myhill-Nerode** para  $L$  si:

1.  $\equiv$  es una **congruencia por la derecha**.
2.  $\equiv$  **refina**  $L$ .
3. El número de clases de equivalencia de  $\equiv$  es **finita**.

A partir de una relación  $\equiv$  de Myhill-Nerode podemos construir un DFA  $\mathcal{A}_\equiv$

$$\mathcal{A} \longrightarrow \equiv_{\mathcal{A}}$$

$$\equiv \longrightarrow \mathcal{A}_\equiv$$

# Construcción del DFA $\mathcal{A}_{\equiv}$ (recordatorio)

Dada una **relación de Myhill-Nerode**  $\equiv$  para  $L \subseteq \Sigma^*$ , definimos el autómata:

$$\mathcal{A}_{\equiv} = (Q_{\equiv}, \Sigma, \delta_{\equiv}, q_{\equiv}, F_{\equiv})$$

$$\blacksquare Q_{\equiv} = \{ [w]_{\equiv} \mid w \in \Sigma^* \} \quad (\text{¿por qué } Q_{\equiv} \text{ es finito?})$$

$$\blacksquare q_{\equiv} = [\epsilon]_{\equiv}$$

$$\blacksquare F_{\equiv} = \{ [w]_{\equiv} \mid w \in L \} \quad (\text{¿por qué } F_{\equiv} \text{ esta bien definida?})$$

$$\blacksquare \delta_{\equiv}([w]_{\equiv}, a) = [wa]_{\equiv} \quad (\text{¿por qué } \delta_{\equiv} \text{ esta bien definida?})$$

Teorema

$$\mathcal{L}(\mathcal{A}_{\equiv}) = L$$

Demostración: ejercicio.

$$\mathcal{A} \longrightarrow \equiv_{\mathcal{A}} \quad \text{y} \quad \equiv \longrightarrow \mathcal{A}_{\equiv} \text{ (recordatorio)}$$

## Teorema

1. Si  $\mathcal{A}$  es un DFA que acepta  $L$  y si construimos:

$$\mathcal{A} \longrightarrow \equiv_{\mathcal{A}} \longrightarrow \mathcal{A}_{\equiv_{\mathcal{A}}}$$

entonces  $\mathcal{A}$  es **isomorfo** (“equivalente”) a  $\mathcal{A}_{\equiv_{\mathcal{A}}}$ .

2. Si  $\equiv$  es una relación de Myhill-Nerode para  $L$  y si construimos:

$$\equiv \longrightarrow \mathcal{A}_{\equiv} \longrightarrow \equiv_{\mathcal{A}_{\equiv}}$$

entonces la relación  $\equiv$  es **equivalente** a  $\equiv_{\mathcal{A}_{\equiv}}$ .

Demostración: ejercicio

# La relación $\equiv_L$ de un lenguaje $L$ (recordatorio)

## Definición

Dado un lenguaje  $L \subseteq \Sigma^*$ , se define la relación de equivalencia  $\equiv_L$  como:

$$u \equiv_L v \quad \text{ssi} \quad (u \cdot w \in L \Leftrightarrow v \cdot w \in L) \quad \forall w \in \Sigma^*$$

¿cuáles son las clases de equivalencia para  $L = (ab)^*$ ?

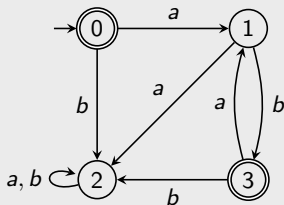
- $[\epsilon]_{\equiv_L} = \{\epsilon, ab, abab, ababab, \dots\} = (ab)^*$
- $[a]_{\equiv_L} = \{a, aba, ababa, abababa, \dots\} = a(ba)^*$
- $[b]_{\equiv_L} = \{b, bb, ba, abb, \dots\} = b\Sigma^* + \Sigma^*(aa + bb)\Sigma^*$



# La relación $\equiv_L$ de un lenguaje $L$ (recordatorio)

1.  $\equiv_L$  es una **congruencia por la derecha**.
2.  $\equiv_L$  **refina**  $L$ .
3. Si  $\equiv$  es una congruencia por la derecha y refina  $L$  entonces  $\equiv$  **refina**  $\equiv_L$ .

Ejemplo  $L = (ab)^*$



$$[\epsilon]_{\mathcal{A}} = \epsilon$$

$$[a]_{\mathcal{A}} = a(ba)^*$$

$$[ab]_{\mathcal{A}} = ab(ab)^*$$

$$[b]_{\mathcal{A}} = b\Sigma^* + \Sigma^*(aa + bb)\Sigma^*$$

# La relación $\equiv_L$ de un lenguaje $L$ (recordatorio)

1.  $\equiv_L$  es una **congruencia por la derecha**.
2.  $\equiv_L$  **refina**  $L$ .
3. Si  $\equiv$  es una congruencia por la derecha y refina  $L$  entonces  $\equiv$  **refina**  $\equiv_L$ .

Ejemplo  $L = (ab)^*$

$$\begin{array}{ll} [\epsilon]_{\mathcal{A}} = \epsilon & [a]_{\equiv_L} = a(ba)^* \\ [a]_{\mathcal{A}} = a(ba)^* & [ab]_{\equiv_L} = (ab)^* \\ [ab]_{\mathcal{A}} = ab(ab)^* & [b]_{\equiv_L} = b\Sigma^* + \Sigma^*(aa + bb)\Sigma^* \\ [b]_{\mathcal{A}} = b\Sigma^* + \Sigma^*(aa + bb)\Sigma^* & \end{array}$$

Notar que:

$$\begin{array}{l} [a]_{\equiv_L} = [a]_{\mathcal{A}} \\ [ab]_{\equiv_L} = [\epsilon]_{\mathcal{A}} \cup [ab]_{\mathcal{A}} \\ [b]_{\equiv_L} = [b]_{\mathcal{A}} \end{array}$$

Por lo tanto,  $\equiv_{\mathcal{A}}$  **refina**  $\equiv_L$

# Teorema de Myhill-Nerode (recordatorio)

## Teorema

Sea  $L \subseteq \Sigma^*$ . Las siguientes propiedades son equivalentes:

1.  $L$  es **regular**.
2. existe una **relación de Myhill-Nerode** para  $L$ .
3. la relación  $\equiv_L$  tiene una cantidad **finita** de clases de equivalencia.

# Teorema de Myhill-Nerode

## Corolarios

1.  $\equiv_L \longrightarrow \mathcal{A}_{\equiv_L}$  produce el autómata con la menor cantidad de estados.
2. Todo autómata  $\mathcal{A}$  tal que  $\equiv_{\mathcal{A}} = \equiv_L$  son **isomorfos** (“equivalentes”).
3. El **algoritmo de minimización** produce un autómata isomorfo a  $\mathcal{A}_{\equiv_L}$ .

## Demostración (punto 3)

Sea  $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$  un autómata que acepta  $L$  ya **minimizado** :

$$\begin{aligned} u \equiv_L v &\iff (u \cdot w \in L \iff v \cdot w \in L) \quad \forall w \in \Sigma^* \\ &\iff (\hat{\delta}(q_0, u \cdot w) \in F \iff \hat{\delta}(q_0, v \cdot w) \in F) \quad \forall w \in \Sigma^* \\ &\iff (\hat{\delta}(\hat{\delta}(q_0, u), w) \in F \iff \hat{\delta}(\hat{\delta}(q_0, v), w) \in F) \quad \forall w \in \Sigma^* \\ &\iff \hat{\delta}(q_0, u) \approx_{\mathcal{A}} \hat{\delta}(q_0, v) \\ &\iff \hat{\delta}(q_0, u) = \hat{\delta}(q_0, v) \quad (\text{¿por qué?}) \\ &\iff u \equiv_{\mathcal{A}} v \end{aligned}$$

# Outline

Teorema de Myhill-Nerode (anterior)

**2DFA**

2DFA vs DFA

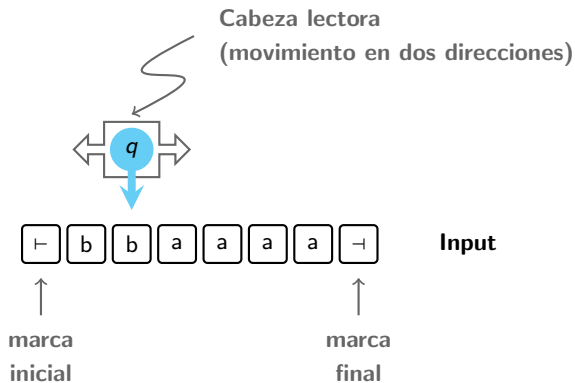
# ¿cuánto se parece un autómata a un algoritmo?

## ¿cuáles son las diferencias?

1. Memoria.
2. “Movimiento” de la máquina.

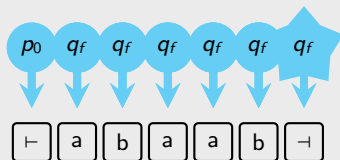
En esta clase, veremos como agregar **movimiento en dos direcciones**

# Autómatas finitos deterministas en **dos direcciones**



# Autómatas finitos deterministas en dos direcciones

## Ejemplo





# Autómatas finitos deterministas en dos direcciones

## Definición

Un autómata finito determinista en 2 direcciones (2DFA) es una estructura:

$$\mathcal{A} = (Q, \Sigma, \vdash, \dashv, \delta, q_0, q_f)$$

- $Q$  es un conjunto finito de estados.
- $\Sigma$  es el alfabeto de input.
- $\vdash$  y  $\dashv$  son las marcas (símbolos) iniciales y finales.
- $\delta : Q \times (\Sigma \cup \{\vdash, \dashv\}) \rightarrow Q \times \{\leftarrow, \rightarrow\}$  es la **función parcial de transición**.
- $q_0$  es el estado inicial.
- $q_f$  es el estado final.

# Autómatas finitos deterministas en dos direcciones

## Ejemplo de función de transición de un 2DFA

|       | $\vdash$             | $a$                  | $b$                  | $\dashv$            |
|-------|----------------------|----------------------|----------------------|---------------------|
| $q_0$ | $(q_0, \rightarrow)$ | $(q_1, \rightarrow)$ | $(q_0, \rightarrow)$ | $(p_0, \leftarrow)$ |
| $q_1$ | -                    | $(q_2, \rightarrow)$ | $(q_1, \rightarrow)$ | -                   |
| $q_2$ | -                    | $(q_0, \rightarrow)$ | $(q_2, \rightarrow)$ | -                   |
| $p_0$ | $(q_f, \rightarrow)$ | $(p_0, \leftarrow)$  | $(p_1, \leftarrow)$  | -                   |
| $p_1$ | -                    | $(p_1, \leftarrow)$  | $(p_0, \leftarrow)$  | -                   |
| $q_f$ | -                    | $(q_f, \rightarrow)$ | $(q_f, \rightarrow)$ | -                   |

# Configuración de un 2DFA

Sea:

- Un 2DFA  $\mathcal{A} = (Q, \Sigma, \vdash, \dashv, \delta, q_0, q_f)$ .
- Una palabra  $w = a_1 a_2 \dots a_n \in \Sigma^*$ .

Defina  $a_0 = \vdash$  y  $a_{n+1} = \dashv$  tal que el input se define como:

$$a_0 a_1 \dots a_n a_{n+1} = \vdash \cdot w \cdot \dashv$$

Una **configuración** de  $\mathcal{A}$  sobre  $w$  viene dado por un par:

$$(q, i) \in Q \times \{0, \dots, n+1\}$$

- $q$  es el **estado actual** del autómata.
- $i$  es la **posición actual** de la cabeza lectora.

# Configuración de un 2DFA

Sea:

- Un 2DFA  $\mathcal{A} = (Q, \Sigma, \vdash, \dashv, \delta, q_0, q_f)$ .
- Una palabra  $w = a_1 a_2 \dots a_n \in \Sigma^*$ .

Se define la relación de **siguiente configuración**  $\xrightarrow{\mathcal{A}}$  de  $\mathcal{A}$  sobre  $w$  como:

$$(p, i) \xrightarrow{\mathcal{A}} (q, j)$$

tal que:

- Si  $\delta(p, a_i) = (q, \rightarrow)$ , entonces  $(p, i) \xrightarrow{\mathcal{A}} (q, i + 1)$ .
- Si  $\delta(p, a_i) = (q, \leftarrow)$ , entonces  $(p, i) \xrightarrow{\mathcal{A}} (q, i - 1)$ .

## ¿cómo ejecuto mi 2DFA?

Sea:

- Un 2DFA  $\mathcal{A} = (Q, \Sigma, \vdash, \dashv, \delta, q_0, q_f)$ .
- El input  $w = a_1 a_2 \dots a_n \in \Sigma^*$ .

Una **ejecución** (o run)  $\rho$  de  $\mathcal{A}$  sobre  $w$  es una secuencia de configuraciones:

$$\rho : (p_0, i_0) \rightarrow (p_1, i_1) \rightarrow \dots \rightarrow (p_m, i_m)$$

- $p_0 = q_0$  y  $i_0 = 0$ .
- $(p_j, i_j) \xrightarrow{\mathcal{A}} (p_{j+1}, i_{j+1}) \quad \forall j \in [0, m-1]$

Una ejecución  $\rho$  de  $\mathcal{A}$  sobre  $w$  es de **aceptación** si:

$$p_m = q_f \quad \text{y} \quad i_m = n + 1$$

# Lenguaje aceptado por un 2DFA

Sea un autómata  $\mathcal{A} = (Q, \Sigma, \vdash, \dashv, \delta, q_0, q_f)$  y  $w \in \Sigma^*$ .

## Definiciones

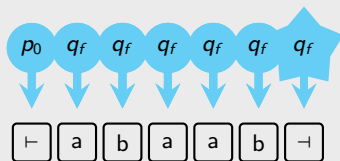
- $\mathcal{A}$  **acepta**  $w$  si hay una ejecución de  $\mathcal{A}$  sobre  $w$  que es de **aceptación**.
- El **lenguaje aceptado por**  $\mathcal{A}$  se define como:

$$\mathcal{L}(\mathcal{A}) = \{w \in \Sigma^* \mid \mathcal{A} \text{ acepta } w\}$$

Un 2DFA puede parar por error o NO parar nunca!

¿cuál es la ejecución de este autómata?

Ejemplo



|       | $\vdash$             | $a$                  | $b$                  | $\neg$              |
|-------|----------------------|----------------------|----------------------|---------------------|
| $q_0$ | $(q_0, \rightarrow)$ | $(q_1, \rightarrow)$ | $(q_0, \rightarrow)$ | $(p_0, \leftarrow)$ |
| $q_1$ | -                    | $(q_2, \rightarrow)$ | $(q_1, \rightarrow)$ | -                   |
| $q_2$ | -                    | $(q_0, \rightarrow)$ | $(q_2, \rightarrow)$ | -                   |
| $p_0$ | $(q_f, \rightarrow)$ | $(p_0, \leftarrow)$  | $(p_1, \leftarrow)$  | -                   |
| $p_1$ | -                    | $(p_1, \leftarrow)$  | $(p_0, \leftarrow)$  | -                   |
| $q_f$ | -                    | $(q_f, \rightarrow)$ | $(q_f, \rightarrow)$ | -                   |

$$L = \{ w \in \Sigma^* \mid (\#a(w) = 0 \bmod 3) \text{ y } (\#b(w) = 0 \bmod 2) \}$$

# Outline

Teorema de Myhill-Nerode (anterior)

2DFA

2DFA vs DFA



## 2DFA vs lenguajes regulares

Para todo lenguaje regular  $L$  existe un 2DFA  $\mathcal{A}$ :

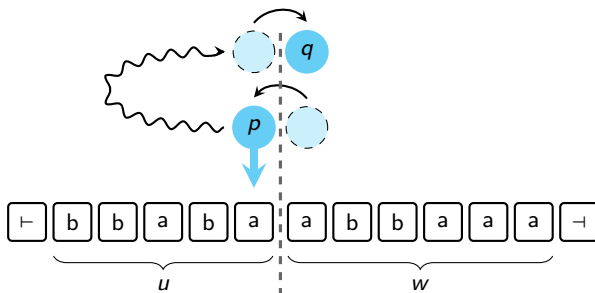
$$L = \mathcal{L}(\mathcal{A})$$

En otras palabras,  $\text{DFA} \subseteq \text{2DFA}$ .

¿són los 2DFA **más poderosos** que los DFA?

Demostraremos que NO!

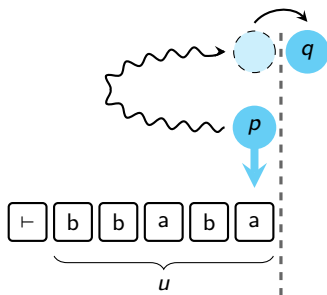
¿cuánta información puede recordar un 2DFA?



*"Cada vez que  $\mathcal{A}$  cruce de  $w$  a  $u$  en el estado  $p$ ,  
 $\mathcal{A}$  cruzará de regreso en el estado  $q$ ."*

este comportamiento solo depende de  $u$  y no de  $w$ !

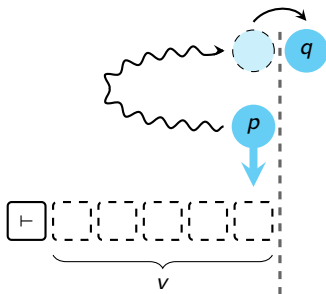
¿cuánta información puede recordar un 2DFA?



Para cada  $u \in \Sigma^*$  definimos la función  $T_u : Q \cup \{\bullet\} \rightarrow Q \cup \{\perp\}$  tal que:

- $T_u(p) = q$  ssi desde  $(p, |u|)$  cruza en la config.  $(q, |u| + 1)$ .
- $T_u(p) = \perp$  ssi desde  $(p, |u|)$  nunca cruza de  $u$ .
- $T_u(\bullet) = q$  ssi desde  $(q_0, 0)$  cruza por 1era vez con  $(q, |u| + 1)$ .
- $T_u(\bullet) = \perp$  ssi desde  $(q_0, 0)$  nunca cruza de  $u$ .

¿cuánta información puede recordar un 2DFA?



Suponga ahora que tenemos una palabra  $v$  tal que:

$$T_v = T_u$$

Entonces,  $v$  es **indistinguible** de  $u$  según  $\mathcal{A}$

En otras palabras,  $u \cdot w \in \mathcal{L}(\mathcal{A}) \Leftrightarrow v \cdot w \in \mathcal{L}(\mathcal{A})$  para todo  $w \in \Sigma^*$ .

# Recordatorio: Teorema de Myhill-Nerode

Sea  $L \subseteq \Sigma^*$  cualquier lenguaje.

## Definición (Relación de Myhill-Nerode)

Una relación de equivalencia  $\equiv$  en  $\Sigma^*$  es de **Myhill-Nerode** para  $L$  si:

1.  $\equiv$  es una **congruencia por la derecha**:

$$u \equiv v \text{ entonces } u \cdot w \equiv v \cdot w \quad \forall w \in \Sigma^*$$

2.  $\equiv$  **refina**  $L$ .

$$u \equiv v \text{ entonces } (u \in L \Leftrightarrow v \in L)$$

3. El número de clases de equivalencia de  $\equiv$  es **finita**.

# Recordatorio: Teorema de Myhill-Nerode

## Definición

Dado un lenguaje  $L \subseteq \Sigma^*$ , se define la relación de equivalencia  $\equiv_L$  como:

$$u \equiv_L v \quad \text{si, y solo si,} \quad (u \cdot w \in L \Leftrightarrow v \cdot w \in L) \quad \forall w \in \Sigma^*$$

## Teorema de Myhill-Nerode

Sea  $L \subseteq \Sigma^*$ . Las siguientes propiedades son equivalentes:

1.  $L$  es **regular**.
2. existe una **relación de Myhill-Nerode** para  $L$ .
3. la relación  $\equiv_L$  tiene una cantidad **finita** de clases de equivalencia.

# ¿cuánta información puede almacenar un 2DFA?

1. La relación  $\equiv_T$  entre palabras en  $\Sigma^*$  tal que:

$$u \equiv_T v \quad \text{si, y solo si,} \quad T_u = T_v$$

es una **relación de equivalencia**.

2.  $\equiv_T$  es una **congruencia por la derecha**. ( $u \equiv_T v \Rightarrow \forall w. u \cdot w \equiv_T v \cdot w$ )
3.  $\equiv_T$  **refina** a  $\mathcal{L}(\mathcal{A})$ . ( $u \equiv_T v \Rightarrow (u \in L \Leftrightarrow v \in L)$ )
4. La relación  $\equiv_T$  tiene una **cantidad finita** de clases de equivalencia.

$$T : Q \cup \{\bullet\} \rightarrow Q \cup \{\perp\}$$

¿cuántas funciones  $T$  existen?

$\equiv_{\mathcal{L}(\mathcal{A})}$  tiene una cantidad **finita** de **clases de equivalencia**.

## 2DFA aceptan solo lenguajes regulares

### Teorema

Para todo 2DFA  $\mathcal{A}$  existe un DFA  $\mathcal{A}'$  tal que:

$$\mathcal{L}(\mathcal{A}) = \mathcal{L}(\mathcal{A}')$$

Por lo tanto, 2DFA  $\equiv$  DFA.

¿cómo construimos el DFA?



# Cierre de clase

En esta clase vimos:

1. Autómatas con movimiento en dos direcciones (2DFA).
2. Demostramos que todos los 2DFA definen lenguajes regulares.
3. Uso del Teorema de Myhill-Nerode.

**Próxima clase:** Algoritmos para evaluación de autómatas.